

WooCommerce ↔ Skipily verbinden (Schritt für Schritt)

Voraussetzung: Lies zuerst `00-Skipily-Anbindung-Grundlagen.md` und halte deinen **Skipily-API-Schlüssel** bereit. WooCommerce ist ein Plugin für WordPress – du brauchst also einen WordPress-Login mit Admin-Rechten.

Wir verbinden beides über das kostenlose No-Code-Werkzeug **Make.com**. Du klickst alles zusammen – kein Programmieren.

Was am Ende funktioniert:

- Neues/geändertes Produkt in WooCommerce → erscheint automatisch bei Skipily.
- Bestellung bei Skipily → landet automatisch in WooCommerce.
- Versand in WooCommerce erfasst → Status + Sendungsnummer gehen zurück an Skipily.

Plane **ca. 45–60 Minuten** für die Ersteinrichtung ein.

 **Abkürzung – fertige Vorlage:** Ein fertiges Make-Template „WooCommerce ↔ Skipily“ gibt es in der öffentlichen Galerie (noch) nicht, aber wir liefern **importierbare Blueprints** mit (Ordner `blueprints/`), die den kompletten Skipily-Teil (URL, Header, JSON) vorbefüllen: `skipily-produkt-anlegen.json` (Teil A), `skipily-bestellung-empfangen.json` (Teil B), `skipily-versand-melden.json` (Teil C). **Import:** Neues Szenario → unten „...“ (**More**) → „**Import Blueprint**“ → Datei wählen. Danach musst du nur noch deinen `x-api-key` einsetzen, den WooCommerce-Baustein ergänzen und die Felder zuordnen – genau das zeigen die Teile A–C unten.

Teil 0 – Vorbereitung (10 Min)

1. **Make.com-Konto** anlegen: <https://www.make.com> → „Get started free“.
2. **WooCommerce-API-Schlüssel** erzeugen (damit Make deinen Shop lesen/schreiben darf):
 - WordPress-Adminbereich → **WooCommerce** → **Einstellungen** → **Erweitert** → **REST-API**.
 - „**Schlüssel hinzufügen**“ klicken.
 - Beschreibung: `Skipily`, Benutzer: dich selbst, **Berechtigung: Lesen/Schreiben**.
 - „**API-Schlüssel erzeugen**“ → es erscheinen ein **Consumer Key** und ein **Consumer Secret**. **Jetzt kopieren und sicher speichern** (das Secret wird nur einmal angezeigt!).
3. **Skipily-API-Schlüssel** aus dem Provider-Portal bereitlegen (siehe Grundlagen).

Jetzt hast du drei Zugangsdaten: WooCommerce-Key, WooCommerce-Secret, Skipily-API-Schlüssel.

Teil A – Produkte von WooCommerce zu Skipily schicken (15 Min)

Ziel: Legst du in WooCommerce ein Produkt an, wird es automatisch an Skipily gemeldet.

1. In **Make.com** oben rechts „**Create a new scenario**“ klicken.
2. Auf das große „+“ klicken → nach „**WooCommerce**“ suchen → Baustein „**Watch Products**“ (Produkte beobachten) wählen.
 - Beim ersten Mal fragt Make nach einer „**Connection**“: „**Add**“ klicken und deine Shop-Adresse (`https://dein-shop.de`), **Consumer Key** und **Consumer Secret** aus Teil 0 eintragen → **Save**.
 - Bei „Watch“ wähle „**Created and updated**“ (neu UND geändert).
3. Rechts neben WooCommerce erneut auf „+“ → „**HTTP**“ → Baustein „**Make a request**“ (eine Anfrage senden). Das ist der Aufruf der Skipily-API.
4. Diesen HTTP-Baustein füllst du so aus:
 - **URL:** `https://vcjwlyqkfkszumdrfvtn.supabase.co/functions/v1/products-api`
 - **Method:** POST
 - **Headers** (auf „Add item“ klicken, je Zeile ein Header):
 - Name `Authorization` – Wert `Bearer eyJhbGci...` (langer öffentlicher Wert aus den Grundlagen)
 - Name `x-api-key` – Wert = **dein Skipily-API-Schlüssel**
 - Name `Content-Type` – Wert `application/json`
 - **Body type:** Raw → **Content type:** JSON (`application/json`)
 - **Request content:** hier baust du die Produktdaten. Klicke in die Felder und wähle per Klick die passenden WooCommerce-Felder aus dem oberen WooCommerce-Baustein (erkennbar an den bunten „Chips“). Vorlage:

```
{
  "name": "{{Produktname aus WooCommerce}}",
  "sku": "{{SKU aus WooCommerce}}",
  "price": "{{Preis aus WooCommerce}}",
  "currency": "EUR",
  "stock_quantity": "{{Lagerbestand aus WooCommerce}}",
  "in_stock": true,
  "description": "{{Kurzbeschreibung aus WooCommerce}}"
}
```

5. **Speichern** (Diskettensymbol unten). Mit „**Run once**“ testest du: Lege in WooCommerce ein Test-Produkt an → das Szenario sollte grün durchlaufen und das Produkt erscheint bei Skipily.

6. Links unten den Schalter „**Scheduling**“ auf **ON** stellen (z. B. alle 15 Min), damit es dauerhaft automatisch läuft.

Tipp – Doppelte vermeiden: Speichere dir die von Skipily zurückgegebene `id` je Produkt (z. B. in einem WooCommerce-„Meta-Feld“). Für spätere Änderungen nutzt du dann **PUT** auf `.../products-api?id=DIE-ID` statt immer neu anzulegen. Für den Start reicht aber das einfache Anlegen.

Teil B – Skipily-Bestellungen in WooCommerce anlegen (15 Min)

Ziel: Kauft jemand über Skipily, soll die Bestellung automatisch in WooCommerce auftauchen. Skipily „klingelt“ dazu bei einer Web-Adresse, die Make dir gibt (**Webhook**).

1. In Make.com **neues Szenario** anlegen.
2. Ersten Baustein „**Webhooks**“ → „**Custom webhook**“ wählen → „**Add**“ → Name z. B. `Skipily-Bestellungen` → Make zeigt dir eine **Adresse (URL)** an → **kopieren**.
3. Diese URL bei Skipily hinterlegen: **Provider-Portal** → **Einstellungen** → **Integrationen** → „**Webhook-URL für Bestellungen**“ → einfügen → speichern. *(Kein Feld gefunden? Schick die URL an support@skipily.app, wir tragen sie ein.)*
4. Zurück in Make: „+“ → „**WooCommerce**“ → „**Create an Order**“ (Bestellung anlegen). Ordne die Felder aus dem Skipily-Webhook zu (Kundenname, Artikel, Menge, Preis, Adresse).
5. **Speichern** und **Scheduling ON**.
6. **Test:** Löse bei Skipily eine Test-Bestellung aus (oder bitte uns darum) → die Bestellung sollte in WooCommerce erscheinen.

Merke dir die Skipily-Bestell-ID aus dem Webhook (Feld `order_id` bzw. `order_number`) – die brauchst du in Teil C für die Versandrückmeldung. Am besten in ein WooCommerce-Bestellnotiz-/Meta-Feld schreiben.

Teil C – Versand & Tracking an Skipily zurückmelden (10 Min)

Ziel: Sobald du in WooCommerce versendest, sieht der Kunde bei Skipily „versandt“ + Sendungsnummer.

1. In Make.com **neues Szenario** anlegen.
2. Erster Baustein „**WooCommerce**“ → „**Watch Orders**“, bei „Watch“ „**Updated**“, und in den Filtern nur Bestellungen mit Status „**completed/shipped**“ durchlassen (Filter zwischen den Bausteinen: Zahnrad/Schraubenschlüssel → Bedingung „Status = completed“).

3. Zweiter Baustein „HTTP“ → „Make a request“:

- **URL:** `https://vcjwlyqkfkszumdrfvtm.supabase.co/functions/v1/orders-api?id={{Skipily-Bestell-ID}}` (die ID aus Teil B – aus dem Bestell-Meta-Feld ziehen)
- **Method:** `PUT`
- **Headers:** dieselben drei wie in Teil A (`Authorization` , `x-api-key` , `Content-Type`).
- **Body** → **Raw** → **JSON:**

```
{
  "status": "shipped",
  "tracking_number": "{{Sendungsnummer aus WooCommerce}}",
  "tracking_url": "{{Tracking-Link, falls vorhanden}}"
}
```

4. **Speichern, Run once** zum Test (eine Bestellung auf „completed“ setzen), dann **Scheduling ON**.

Fertig! Kurz-Kontrolle

- ☐ Test-Produkt in WooCommerce → erscheint bei Skipily (Teil A)
- ☐ Test-Bestellung bei Skipily → erscheint in WooCommerce (Teil B)
- ☐ Bestellung auf „versendet“ → Skipily zeigt „shipped“ + Tracking (Teil C)

Wenn etwas klemmt

- **„401/403“ in Make:** Ein Header stimmt nicht – prüfe `x-api-key` (dein Skipily-Schlüssel) und den `Authorization`-Wert (exakt kopieren, keine Leerzeichen).
- **„400 – name required“:** Im Produkt-Body fehlt `name` . Feld neu zuordnen.
- **Nichts passiert:** Steht der **Scheduling-Schalter auf ON**? Läuft in Make links das Szenario „aktiv“ (grün)?
- **Hilfe:** support@skipily.app – gerne mit Screenshot der Make-Fehlermeldung.